

Full-Stack Developer Technical Case Study

Mini Operations ERP

Objective

Build a small but production-oriented **full-stack Operations ERP**.

This is not a UI-only assignment. We want to evaluate your ability to build and connect:

- Frontend
 - Backend APIs
 - Relational database
 - Authentication
 - Business logic
 - Transactions
 - Validation
 - Testing
 - Portable architecture
-

Business Scenario

A company operates from multiple locations and manages inventory, work orders, internal stock transfers, and customer orders.

You need to build a working system covering the following flow:

Inventory → Work Order → Stock Check → Internal Transfer / Shortage → Customer Reservation

Mandatory Modules

1. Authentication & Roles

Create login with minimum roles:

- Admin
- Operations User
- Sales User

Backend authorization is mandatory.

Example:

- Admin can create Work Orders.
 - Operations User can manage inventory and transfers.
 - Sales User can create orders and reserve stock.
-

2. Inventory Management

Inventory should include:

- Item
- Category
- Location
- Batch
- Physical Quantity
- Reserved Quantity
- Available Quantity

Available Quantity should be calculated correctly.

Example:

Physical = 100
Reserved = 30
Available = 70

System must prevent:

- Negative inventory
 - Invalid quantity
 - Duplicate inventory transaction
 - Reservation beyond available quantity
-

3. Work Order + Material Stock Check

Admin can create a Work Order.

Minimum fields:

- Work Order ID
- Location
- Item
- Required Quantity
- Assigned User
- Status

Statuses:

- Assigned
- In Progress
- Completed

A Work Order may require material.

Example:

Required Material = 100
Available at Location = 60
Shortage = 40

The system should calculate the shortage automatically.

4. Internal Stock Transfer

If the required material is available at another location, allow an Internal Transfer.

Fields:

- Transfer ID
- Source Location
- Destination Location
- Item
- Quantity
- Status

Statuses:

- Requested
- Dispatched
- Received

Important rule:

On Dispatch

Source inventory reduces.

Before Receipt

Destination inventory must NOT increase.

On Receipt

Destination inventory increases.

The system must prevent the same transfer from being received twice.

5. Customer Order & Stock Reservation

Sales User should be able to create an order.

Example:

Item A
Available = 100
Customer Order = 60

After reservation:

Physical = 100
Reserved = 60
Available = 40

Two users must not be able to reserve more stock than actually exists.

Example:

Available = 100

User A → Reserve 80
User B → Reserve 50

Both requests must **not** succeed.

Solve this correctly at backend/database level.

Technical Requirements

Your application must have:

- Frontend connected with backend APIs
- Relational database
- Proper data relationships
- Backend validation
- Error handling
- Authentication
- Role-based authorization
- Database transactions where required

- Environment-based configuration

The business logic should not depend heavily on one hosting provider or environment.

Minimum Screens

Build only these screens:

1. Login
2. Inventory
3. Work Orders
4. Internal Transfers
5. Customer Orders

Focus on functionality instead of excessive UI design.

Mandatory Tests

At minimum implement these tests:

Test 1

Cannot reserve more than available inventory.

Test 2

Cannot transfer more than available inventory.

Test 3

Destination stock increases only after transfer receipt.

Test 4

Same transfer cannot be received twice.

Test 5

Unauthorized user cannot perform restricted operation.

Submission Requirements

Submit:

1. Git Repository

Complete source code.

2. README

Include:

- Tech stack
- Project setup
- Database setup
- Environment variables
- How to run
- How to test

3. Database Schema / ER Diagram

4. API Documentation

Swagger, Postman or equivalent.

5. Short Demo Video

Maximum **5–7 minutes**.

Show:

Login → Inventory → Work Order → Transfer → Order Reservation

Important Git Requirement

Do not submit the whole application as one final commit.

Your repository should show reasonable development history.

AI Usage

AI tools may be used.

However, you must understand everything submitted.

In the next round, you may be asked to:

- Explain your database
- Modify an API
- Change a business rule

- Debug a failure
- Add a new field
- Fix a transaction issue

If you cannot modify or explain your own application, the submission will not qualify.

Live Verification

Shortlisted candidates will receive one small unannounced change.

Example:

Change 1

Add **DAMAGED** .

Damaged stock should automatically reduce available stock.

OR

Change 2

Allow a transfer to be partially received.

OR

Change 3

Cancel an order and correctly release its reserved inventory.

OR

Change 4

Restrict users to only their assigned location.

Evaluation

Parameter	Marks
Backend & APIs	20
Database Design	15
Business Logic	20
Inventory / Transaction Correctness	15

Parameter	Marks
Frontend Integration	10
Authentication & Authorization	8
Testing	5
Code Quality	5
Documentation	2
Total	100
